

CHAPTER III

FRAMEWORK AND METHODOLOGY

Chapter 3 presents the framework and methodology used to design, develop, and evaluate the proposed blockchain-enabled LoRaWAN agricultural monitoring prototype. It discusses the study's conceptual framework, research design, prototype architecture and experimental setting, development and data collection procedures, security boundary and threat model, evaluation methods, data analysis, and ethical considerations. Together, these sections explain how the prototype will be implemented and systematically tested under normal, connectivity-interruption, and selected security-attack conditions.

3.1 Conceptual Framework

The Input–Process–Output (IPO) model was used as the conceptual framework because the study focuses on the design, operation, and evaluation of a technological prototype. The inputs consist of agricultural sensor data, LoRaWAN network events, system loads, and normal and attack test conditions. These inputs are processed through LoRaWAN transmission, Raspberry Pi edge computing, ChirpStack, MQTT, Node-RED, PostgreSQL, Hyperledger Fabric verification, and Grafana monitoring. The resulting outputs are evaluated in terms of system performance, security effectiveness, data integrity, traceability, and resilience. The IPO framework therefore provides a clear and logical representation of how the system receives data, processes and protects them, and produces measurable outcomes.

Figure 1 shows the study framework using the Input–Process–Output model. In the first step, the inputs consist of agricultural sensor data, LoRaWAN network events, and predefined test conditions. The sensors generate environmental and soil measurements, while network events represent normal transmissions, connectivity interruptions, unauthorized requests, replay attempts, and altered data. Normal and

attack conditions are included so that the prototype can be evaluated not only during regular operation but also when exposed to selected security threats. These inputs were chosen because they represent the actual data, communication behavior, and operating conditions that the proposed agricultural monitoring system is expected to handle.

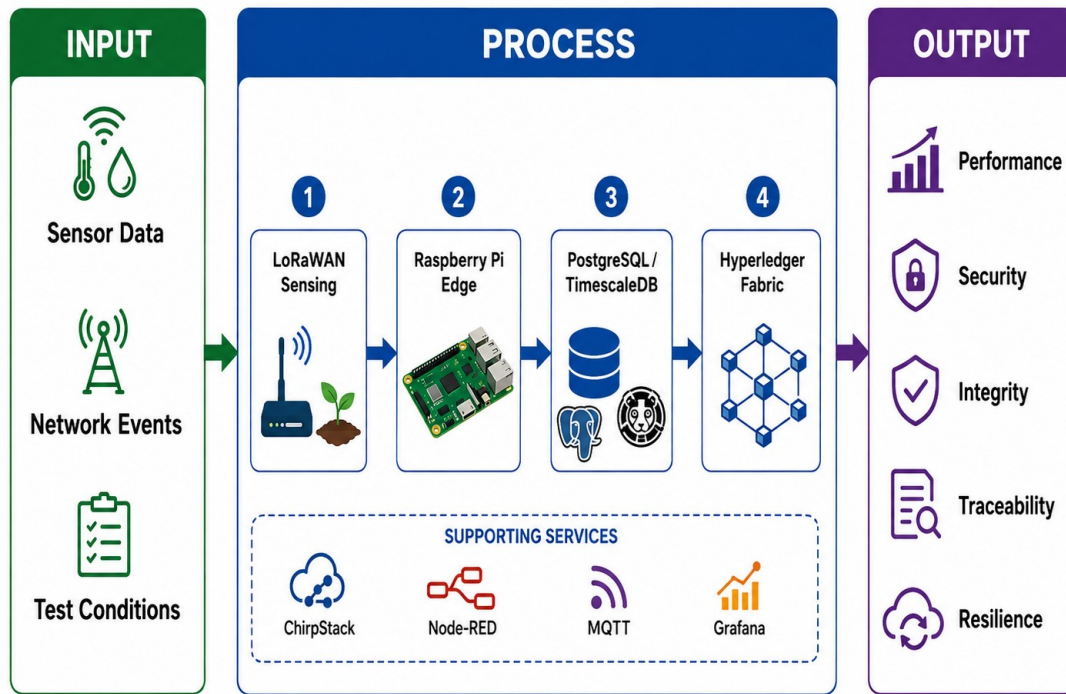


Figure 1. Conceptual Framework of the Study

In the process stage, sensor readings are first transmitted through LoRaWAN because it provides long-range communication with low power consumption, making it suitable for agricultural areas. A Raspberry Pi functions as the edge gateway and runs ChirpStack, MQTT, Node-RED, and local processing services. ChirpStack manages LoRaWAN devices and packets, MQTT supports lightweight data exchange, and Node-RED controls data flow, validation, alerts, and automation. PostgreSQL stores structured information such as devices, locations, users, and alerts. Grafana retrieves the stored readings and system metrics to provide real-time dashboards and monitoring. Hyperledger Fabric records the hashes and essential metadata of the sensor records,

rather than the complete readings, to support permissioned access, integrity verification, traceability, and auditability without placing excessive storage demands on the blockchain.

The output stage represents the measurable results produced by the prototype. System performance is assessed through packet delivery, end-to-end latency, transaction success, and resource utilization. Security effectiveness is examined through the system's ability to detect or reject unauthorized access, replayed data, altered records, and post-storage tampering. Data integrity is confirmed by comparing the hash of a stored reading with the corresponding hash recorded in Hyperledger Fabric, while traceability is evaluated through the availability of timestamps, device identities, transaction records, and audit histories. System resilience is also examined by determining whether local data collection, storage, monitoring, and later synchronization can continue during temporary Internet or network interruptions.

3.2 Methodology

3.2.1 Research Design

This study used a design and development research approach supported by experimental prototype testing. This approach was appropriate because the study involved creating, integrating, and evaluating a working Hyperledger Fabric-based security prototype for LoRaWAN IoT data in a smart banana plantation. Similar studies developed and experimentally tested LoRa-blockchain and agricultural IoT prototypes to assess system functionality, data verification, access control, and transaction performance [25], [26], [31]. In this study, the prototype was designed to collect sensor readings, transmit and process the data, store complete records in PostgreSQL, and record their hashes in Hyperledger Fabric for verification. After development, the system was tested to determine whether it could reliably complete the data flow, verify stored

records, detect post-storage changes, and process blockchain transactions with acceptable prototype-level performance.

3.2.2 Prototype Architecture and Experimental Setting

The study was conducted in a banana plantation in Tagum City, Davao del Norte, Mindanao, Philippines. The experimental testbed used agricultural sensors to collect soil and environmental readings. Figure 2 shows the components of the WisBlock Agriculture Kit used to develop the sensor nodes. The kit includes soil-moisture probes, UV-light sensors, barometric-pressure sensors, ambient-light sensors, environmental sensors for temperature and humidity, and rain sensors. Its RAK4631 core module processes the readings and sends them through LoRaWAN, while the WisBlock baseboards connect and power the sensor modules. The kit also provides RS485, 4–20 mA, and SDI-12 interfaces, allowing other agricultural sensors to be added as the prototype expands. Although an optional cellular module is included, it was not used because the sensor nodes transmitted their readings through LoRaWAN to the study's Raspberry Pi gateway.

Figure 3 shows the Raspberry Pi–RAK5146 LoRaWAN edge gateway used in the prototype. The gateway was assembled by mounting the RAK5146 concentrator on its Pi HAT and connecting the assembly to the Raspberry Pi 4, together with the LoRa and optional GPS antennas. Raspberry Pi OS Lite was installed on a microSD card, after which the system was updated and secured by configuring SSH, disabling root login, and enabling the firewall. Docker was then installed, and the Raspberry Pi interfaces required by the concentrator—SPI, I²C, and the serial port—were enabled. The gateway EUI was generated from the Raspberry Pi network interface, while the LoRa gateway software was deployed through Docker and configured to forward uplink and downlink packets to the ChirpStack network server. Because the prototype was deployed

in the Philippines, the gateway frequency plan was set to AS923, rather than the European 868 MHz configuration shown in the reference setup.

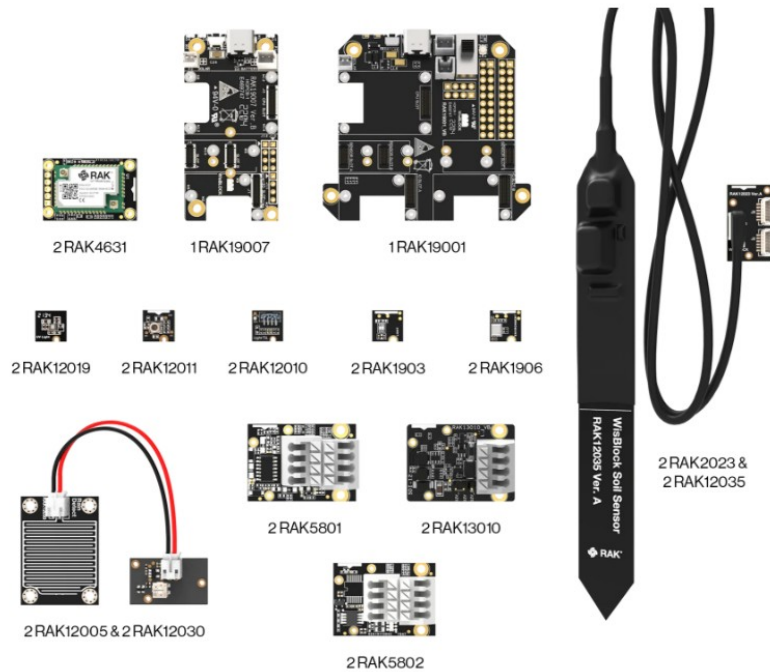


Figure 2. Agricultural Sensors and IoT Components Used in the Prototype

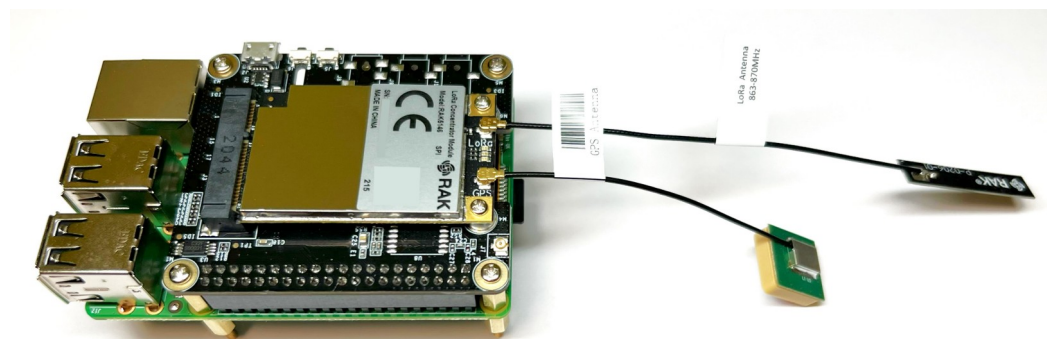


Figure 3. Raspberry Pi-Based LoRaWAN Edge Gateway Used in the Prototype

ChirpStack was installed locally on the Raspberry Pi through Docker Compose and configured as the LoRaWAN network server for the AS923 frequency plan. The RAK5146 concentrator forwarded uplink and downlink packets to the local ChirpStack Gateway Bridge, while the gateway and sensor nodes were registered using their gateway EUI, device EUI, application EUI, and activation keys. ChirpStack then published the decoded sensor readings as JSON messages to a locally hosted Mosquitto MQTT broker. Node-RED subscribed to the corresponding MQTT topics and was configured to validate, organize, timestamp, and route the readings, generate alerts when required, and send the processed data to the storage and verification layers. Because these services operated within the Raspberry Pi and local network, sensor collection and processing did not depend on a remote cloud platform.

PostgreSQL with the TimescaleDB extension was configured as the local operational database, where complete sensor readings, device information, timestamps, locations, and system events were stored. TimescaleDB hypertables supported the growing time-series dataset, while Grafana was connected directly to PostgreSQL through a restricted read-only account to display sensor readings and system-performance indicators. Hyperledger Fabric was deployed through Docker as a permissioned network within the local testbed, and a local application interface generated a cryptographic hash for each stored sensor record and submitted the hash, device identity, timestamp, and essential metadata to the Fabric chaincode. All components communicated through a private Docker and local-area network using defined service names, credentials, and required ports, allowing the database, dashboard, and integrity-verification functions to remain locally available during temporary Internet interruptions.

Figure 4 presents the overall architecture of the proposed LoRaWAN–edge–blockchain prototype. Agricultural sensor nodes transmit readings through LoRaWAN to the Raspberry Pi–RAK5146 edge gateway, which communicates with the locally deployed ChirpStack network server. ChirpStack publishes the decoded sensor data through MQTT to Node-RED for validation, processing, and routing. The complete sensor records are stored in PostgreSQL/TimescaleDB, while a SHA-256 hash representing each record is submitted to Hyperledger Fabric to support data-integrity verification and traceability. The local deployment of these components allows sensor collection, processing, storage, and verification to continue within the edge environment without depending on a remote cloud platform.

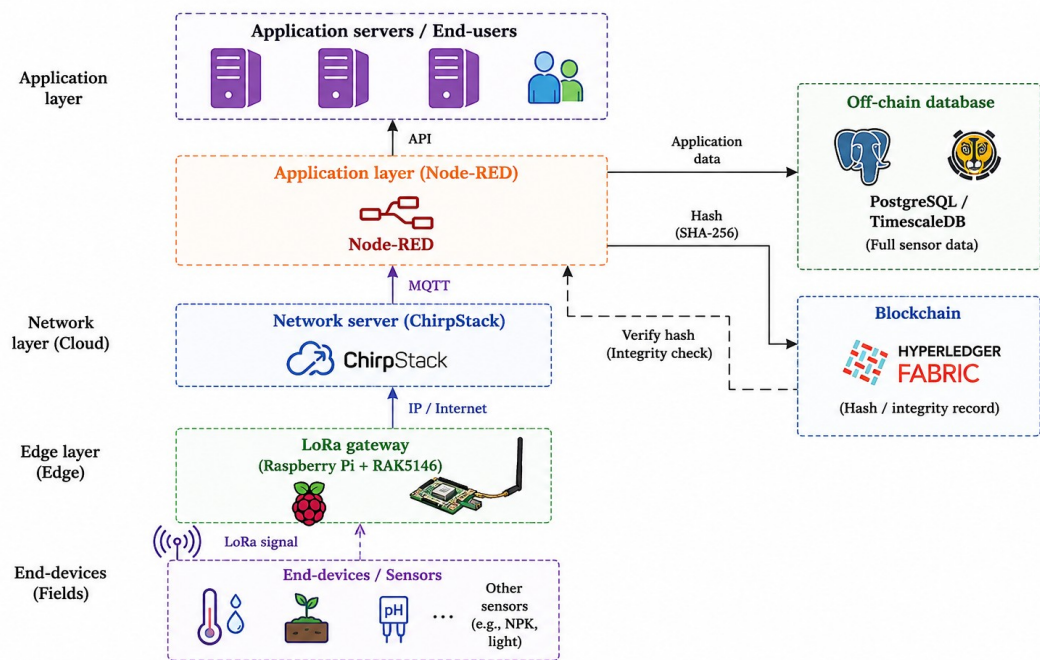


Figure 4. Proposed LoRaWAN–Edge–Blockchain Architecture for Secure Agricultural Monitoring

3.2.3 Prototype Development and Data Collection Procedure

The prototype was developed in stages, beginning with the assembly and testing of the agricultural sensor nodes and Raspberry Pi–RAK5146 LoRaWAN gateway. The required software services were then installed, configured, and integrated, including ChirpStack, MQTT, Node-RED, PostgreSQL/TimescaleDB, Grafana, and Hyperledger Fabric. After the gateway and sensor devices were registered, pilot tests were conducted to verify the complete data flow from sensor transmission and edge processing to database storage, dashboard display, and blockchain hash recording. Once stable operation was confirmed, the prototype collected data under normal operating conditions, temporary connectivity interruptions, and selected security-test scenarios. System logs, sensor records, blockchain transactions, network measurements, and resource-usage data were retained for subsequent performance and security analysis.

3.2.4 Normal-Operation Performance Test Procedure

A normal-operation performance test will be conducted to establish the baseline performance of the prototype before the security and resilience tests are performed. During this test, only a registered agricultural sensor and authorized system components will be used; no invalid traffic, simulated attacks, Internet interruptions, or service failures will be introduced. The sensor will transmit one reading every 15 seconds for 30 minutes, producing approximately 120 expected transmissions per run, consistent with the baseline period used in the resilience experiment. The test will be repeated three times under the same configuration to obtain consistent baseline measurements.

During each run, the complete data flow from the sensor to the LoRaWAN gateway, ChirpStack, Mosquitto MQTT, Node-RED,

PostgreSQL/TimescaleDB, and Hyperledger Fabric will be monitored. ChirpStack and sensor transmission records will be used to determine packet delivery; timestamps generated during sensor transmission and database storage will be used to measure end-to-end latency; Node-RED and Hyperledger Fabric logs will be used to determine blockchain transaction success; and PostgreSQL/TimescaleDB records will be used to determine the number of successfully processed sensor readings. Raspberry Pi CPU and memory utilization will be collected from system and Docker resource-monitoring logs throughout each run. The study already identifies system logs, sensor records, blockchain transactions, network measurements, and resource-usage data as the sources for subsequent performance analysis.

Packet-delivery rate (PDR) will represent the percentage of sensor packets successfully received by the LoRaWAN system relative to the number transmitted by the sensor. The transmitted packets will be identified using the sensor transmission log or LoRaWAN frame counter, while successfully received packets will be identified from ChirpStack uplink records. Duplicate packets will not be counted as additional successful deliveries. PDR will be calculated as:

$$PDR(\%) = \frac{N_{\text{received}}}{N_{\text{transmitted}}} \times 100$$

where N_{received} is the number of unique legitimate packets successfully received by ChirpStack and $N_{\text{transmitted}}$ is the number of packets transmitted by the sensor during the test period.

End-to-end latency will measure the time required for a legitimate sensor reading to travel from sensor transmission until successful storage in PostgreSQL/TimescaleDB. A transmission timestamp associated with each

sensor reading will be matched with its corresponding database-storage timestamp using the device identifier and frame counter or record identifier. End-to-end latency for each successfully delivered reading will be calculated as:

$$L_i = T_{\text{storage},i} - T_{\text{transmission},i}$$

where L_i is the latency of sensor reading i , $T_{\text{transmission},i}$ is its transmission time, and $T_{\text{storage},i}$ is the time at which the corresponding record is successfully stored in the database. The methodology already defines end-to-end latency as the time from legitimate sensor transmission to database storage. Mean latency will be calculated as:

$$\bar{L} = \frac{\sum_{i=1}^n L_i}{n}$$

and will be reported in milliseconds or seconds together with the standard deviation and observed minimum and maximum values.

Transaction success rate will measure whether the hashes and selected metadata submitted to Hyperledger Fabric are successfully committed to the blockchain. A transaction will be considered successful when the submission receives a valid transaction identifier and is confirmed as committed to the ledger without an error or invalid status. The transaction success rate will be calculated as:

$$TSR(\%) = \frac{N_{\text{committed}}}{N_{\text{submitted}}} \times 100$$

where $N_{\text{committed}}$ is the number of valid transactions successfully committed to Hyperledger Fabric and $N_{\text{submitted}}$ is the total number of blockchain transactions submitted during the test.

System throughput will represent the rate at which legitimate sensor records successfully complete the normal processing flow. A record will be considered successfully processed when it passes Node-RED validation and is stored in PostgreSQL/TimescaleDB. Throughput will be calculated as:

$$\text{Throughput} = \frac{N_{\text{processed}}}{T}$$

where $N_{\text{processed}}$ is the number of legitimate sensor records successfully processed and T is the duration of the observation period. Because the sensor generates readings at relatively low frequency, throughput will be reported as **successfully processed records per minute**. This is consistent with the study's definition of valid-message throughput as the number of legitimate sensor readings successfully processed per unit of time.

Raspberry Pi CPU and memory utilization will be monitored throughout each experimental run to determine the computational resources required when ChirpStack, Mosquitto MQTT, Node-RED, PostgreSQL/TimescaleDB, Grafana, and Hyperledger Fabric are operating concurrently. CPU and memory measurements will be obtained from Raspberry Pi system-monitoring and Docker resource logs, as specified in the experimental procedures. For n observations, average CPU utilization will be calculated as:

$$\overline{CPU}(\%) = \frac{\sum_{i=1}^n CPU_i}{n}$$

while maximum CPU utilization will be the highest observed value during the run. Memory utilization will be expressed as the percentage of available Raspberry Pi memory being used:

$$Memory\ Utilization(\%) = \frac{Memory_{used}}{Memory_{total}} \times 100$$

Average and maximum memory utilization will also be reported.

For each of the three normal-operation runs, packet-delivery rate, transaction success rate, throughput, end-to-end latency, CPU utilization, and memory utilization will be recorded. The mean and standard deviation of the continuous performance measures will be reported across the runs, while packet-delivery and transaction-success results will be summarized using frequencies and percentages. These measurements will serve as the baseline performance of the prototype, against which changes observed during the subsequent security, flooding, and resilience tests can be compared.

3.2.5 Security Boundary, Threat Model, and Test Scenarios

The security of the prototype will be evaluated through a multilayer security assessment covering the device and LoRaWAN communication layer, edge and network layer, application and access-control layer, storage layer, and blockchain layer. This approach recognizes that threats may occur at different points in the data flow, from sensor transmission and gateway processing to database storage and blockchain verification. The selected tests, affected layers, evaluation procedures, measures, and expected secure behavior are summarized in Table 5.

Table 5. Proposed Security Assessment and Test Plan for the prototype

Security area	Proposed test	Main layer
---------------	---------------	------------

Authentication and access	Unregistered device, invalid user, or unauthorized Fabric identity	Device, application and blockchain
Replay or spoofing	Resend previously captured packets or duplicate sensor records	LoRaWAN and application
Data integrity	Alter data during processing or after PostgreSQL storage	Application and storage
Traceability	Retrieve the device ID, timestamp, hash, and blockchain transaction history	Storage and blockchain
DoS or flooding	Send increasing invalid or unauthorized traffic	LoRaWAN, edge and application
Resilience	Interrupt Internet access or stop selected local services and measure recovery	Edge and network

Note: DOS: Denial-of-service

3.2.6 Prototype Evaluation and Data Analysis

3.2.6.1 Authentication and Access-Control Test Procedure

The authentication and access-control test will determine whether the prototype accepts only registered devices and authorized system users while rejecting invalid credentials and prohibited actions. The proposed method combines the invalid-device test of Hou et al. [40], the allow-or-deny test of Iftexhar et al. [41], and the unauthorized blockchain-access test of Homaei et al. [47]. Unlike these earlier studies, the proposed test will examine authentication and authorization at four points in the data flow: LoRaWAN device activation, MQTT messaging, PostgreSQL/TimescaleDB storage, and Hyperledger Fabric transactions.

The test will determine whether the prototype accepts registered devices and authorized identities while rejecting invalid credentials and prohibited actions. Testing will be conducted on an isolated network using synthetic sensor data. Logs from ChirpStack, Mosquitto MQTT,

Node-RED, and Hyperledger Fabric will be enabled before the test. Each condition will be repeated 10 times, giving a total of 90 test attempts. Ten repetitions are proposed because the test evaluates the consistency of deterministic allow-or-deny controls rather than estimating behavior in a large population.

Table 6. Authentication and Access-Control Test Design

Test area	Test condition	Number of trials	Measure	Expected result	Evidence collected
LoRaWAN device authentication	Registered device with correct OTAA credentials	10	Join decision and response time	Device joins and transmits data	ChirpStack join event, JoinAccept and accepted uplink
	Registered DevEUI with incorrect AppKey	10	Rejection decision and unauthorized data received	Join request is rejected	ChirpStack event and absence of JoinAccept
	Unregistered DevEUI	10	Rejection decision and unauthorized data received	Device is not activated	ChirpStack event and absence of accepted data
MQTT authentication and authorization	Authorized account publishes to an allowed topic	10	Publish decision and response time	Message is accepted and received by Node-RED	Mosquitto and Node-RED logs
	Account uses an incorrect password	10	Connection decision and response time	Connection is rejected	Mosquitto authentication error
	Limited account publishes to a prohibited topic	10	Publish decision and unauthorized message received	Connection may succeed, but publishing is denied	Mosquitto access-control log and absence of message in Node-RED
Hyperledger Fabric authorization	Authorized writer submits a sensor hash	10	Transaction decision and response time	Transaction is endorsed and committed	Transaction ID, chaincode and peer logs
	Valid identity without writer	10	Rejection decision and	Transaction is rejected	Authorization error and unchanged world

	permission submits a hash		state change		state
	Untrusted or invalid identity submits a transaction	10	Identity decision and state change	Connection or transaction is rejected	Fabric identity or certificate error

LoRaWAN device-authentication procedure

Three sensor configurations will be prepared: one registered device with the correct DevEUI, JoinEUI and AppKey; one device using a registered DevEUI but an incorrect AppKey; and one device using an unregistered DevEUI. For each trial, the sensor will be restarted and instructed to perform an OTAA join. ChirpStack will be observed to determine whether the JoinRequest is received and whether a JoinAccept is generated. When the authorized device joins successfully, it will send one test reading to confirm that the data reaches MQTT. Each of the three conditions will be repeated 10 times. A 30-second interval will be provided between attempts to avoid overlapping join procedures.

MQTT authentication and authorization procedure

Two MQTT accounts will be configured. The authorized Node-RED account will be permitted to access the required sensor topics, while a limited test account will be prohibited from publishing to protected topics. Anonymous access will be disabled. From a separate test laptop, the researcher will attempt an authorized publish, a connection using an incorrect password, and a publish using valid credentials but a prohibited topic. Each condition will be repeated 10 times using `mosquitto_pub`. Mosquitto and Node-RED logs will be checked to confirm that denied messages did not enter the processing pipeline.

Hyperledger Fabric authorization procedure

Three Fabric identities will be prepared: an authorized writer, a valid user without writing permission, and an invalid or untrusted identity. The authorized writer will submit a synthetic sensor hash and metadata through the chaincode. The other two identities will attempt to invoke the same function. Each condition will be repeated 10 times. Before and after every unauthorized attempt, the corresponding blockchain record will be queried to confirm that the world state did not change.

Measures and Data Analysis

For every trial, the researcher will record the expected decision, actual decision, response time, system message or error, and whether the request caused any unauthorized message or blockchain state change. The assessment will use the following measures:

- **Authorized success rate:** percentage of valid requests correctly accepted.
- **Unauthorized rejection rate:** percentage of invalid or prohibited requests correctly rejected.
- **False acceptance:** an unauthorized request that is incorrectly accepted.
- **False rejection:** an authorized request that is incorrectly rejected.
- **Correct-decision rate:** percentage of all requests correctly classified.
- **Response time:** time required for the system to accept, reject, or commit the request.
- **Unauthorized state change:** whether a prohibited request created or modified a protected record.

Response time will be measured separately at each layer. For LoRaWAN, it will be measured from the recorded JoinRequest to the JoinAccept or final rejection decision. For MQTT, it will be measured from the connection or publish request to the broker response. For Hyperledger Fabric, it will be measured from transaction submission to commitment or rejection.

The results will be reported separately for the three layers because each uses a different authentication mechanism. Frequencies and percentages will be used for allow-or-deny outcomes, while the mean and standard deviation will summarize response time. The main security requirement will be zero false acceptance and zero unauthorized state change. Authorized communication failures caused by packet loss will be identified through ChirpStack and gateway logs and will not automatically be classified as authentication failures.

3.2.6.2 Replay or Spoofing Test Procedure

A replay attack occurs when a previously accepted message is captured and transmitted again to make the system process the same data more than once. Spoofing occurs when an unauthorized device creates a message that appears to come from a legitimate sensor. Few related systems experimentally evaluated these threats. Homaei et al. [47] inserted replayed and fabricated sensor readings into a water-consumption dataset and used an LSTM Autoencoder and Isolation Forest to identify them before blockchain storage. However, their tests were conducted at the data-processing layer rather than through actual LoRaWAN packet transmission. A recent LoRaWAN study by Pourghasem et al. [56] also evaluated spoofing detection using device and physical-layer attributes such as RSSI, battery level, and altitude, but its evaluation was dataset-based rather than a live rogue-device attack.

Following the controlled-injection approach of these studies, the proposed prototype will conduct two practical LoRaWAN-level tests: uplink-frame replay and device impersonation through a forged uplink. Testing will be performed only within the isolated experimental network using the authorized sensor node and a separate programmable LoRa test node. Frame-counter validation will remain enabled in ChirpStack. Each test will include 10 legitimate control transmissions and 10 attack transmissions, producing a total of 40 attempts.

Table 7. Replay and Spoofing Test Design

Test	Test condition	Trials	Main measures	Expected result	Evidence collected
LoRaWAN replay	New legitimate uplink	10	Packet acceptance, downstream propagation and processing time	Packet is accepted and reaches the application	Gateway, ChirpStack, MQTT and LoRaWAN Analyzer records
	Previously accepted uplink retransmitted with an old frame counter	10	Replay rejection and unauthorized propagation	Packet is rejected and does not reach MQTT, the database or Fabric	Gateway frame log, ChirpStack error event and downstream records
LoRaWAN spoofing	Genuine uplink from the registered sensor	10	Packet acceptance, downstream propagation and processing time	Packet is authenticated and processed	Gateway, ChirpStack, MQTT and database records
	Forged uplink using the legitimate device address but an invalid MIC	10	Spoofing rejection and unauthorized propagation	Packet fails authentication and creates no application or storage record	Gateway frame log, MIC error, MQTT, database and Fabric records

LoRaWAN Replay Test

The registered agricultural sensor will first transmit a legitimate uplink containing a synthetic sensor reading. The raw LoRaWAN physical payload, device address, frame counter, frequency, data rate, and time of reception will be recorded from the gateway and ChirpStack frame logs. The legitimate packet must appear as an accepted uplink in ChirpStack and must reach MQTT and Node-RED.

The legitimate sensor will then transmit at least three additional uplinks so that its frame counter advances. Afterward, the previously captured physical payload containing the older frame counter will be retransmitted through the programmable LoRa test node. The test node will use the same AS923 frequency, spreading factor, bandwidth, and coding rate as the captured transmission.

For a trial to be considered valid, the gateway must record that it received the retransmitted radio frame. If the gateway does not receive it, the trial will be repeated because this would represent a transmission failure rather than successful replay rejection. Once received, ChirpStack should identify that the frame counter has already been processed and prevent the frame from producing a device uplink event. ChirpStack documents that a previously seen uplink frame counter produces a retransmission error and is not forwarded as a new application uplink when frame-counter checking is enabled.

After every attempt, the researcher will verify that the replayed message did not appear in:

- the ChirpStack application uplink events;
- the MQTT sensor topic;
- Node-RED's accepted-message flow;

- PostgreSQL/TimescaleDB; or
- the Hyperledger Fabric transaction records.

The procedure will be repeated using 10 newly captured legitimate packets and 10 corresponding replay attempts.

LoRaWAN Spoofing Test

The spoofing test will simulate a second device attempting to send data using the identity of the legitimate agricultural sensor. The registered sensor will first transmit 10 genuine uplinks as the control condition. Each transmission must be received by the gateway, authenticated by ChirpStack, forwarded to MQTT, and stored through the normal processing flow.

For the spoofing condition, the programmable LoRa test node will create an uplink containing the current device address of the legitimate sensor and a fabricated sensor value. However, the test node will not possess the legitimate sensor's network session key. Therefore, the message integrity code generated by the test node will not match the legitimate device's credentials.

The forged frame will be transmitted using a valid AS923 radio configuration so that it can be received by the RAK5146 gateway. LoRaWAN uses session keys and a message integrity code to authenticate radio transmissions; when the session key is incorrect, ChirpStack cannot validate the MIC. The gateway may therefore receive the radio signal, but ChirpStack should reject the message before it is published to the application.

Each spoofing trial will be checked at three points:

1. The gateway frame log must confirm that the radio frame was received.
2. ChirpStack must report an authentication or MIC error and must not create an accepted device uplink.

3. MQTT, PostgreSQL/TimescaleDB, and Hyperledger Fabric must contain no record originating from the forged packet.

Ten forged uplinks will be transmitted. A spoofing attempt that is not received by the gateway will not be counted and will be repeated.

Measures and Data Analysis

For every trial, the researcher will record the gateway reception result, ChirpStack decision, downstream propagation, processing time, and occurrence of an unauthorized database or blockchain record. The following measures will be reported:

- **Replay-rejection rate:** percentage of received replayed frames prevented from reaching the application.
- **Spoofing-rejection rate:** percentage of received forged frames rejected because they could not be authenticated.
- **False-acceptance count:** number of replayed or forged frames incorrectly accepted.
- **False-rejection count:** number of genuine frames incorrectly rejected.
- **Unauthorized propagation rate:** percentage of attack frames that reached MQTT, Node-RED, PostgreSQL/TimescaleDB, or Hyperledger Fabric.
- **Processing time:** time from gateway reception until ChirpStack acceptance or rejection, when matching timestamps are available.

LoRaWAN Analyzer will support the assessment by displaying frame-counter sequences and transmission events, while ChirpStack, MQTT, database, and Fabric logs will provide evidence of whether the packet moved to later layers. ChirpStack's gateway-level frame logging can include frames that fail MIC or frame-counter checks, making it possible to confirm that a packet was received by the gateway but rejected before application processing.

The main security requirement will be zero false acceptance and zero unauthorized propagation. All legitimate control messages should be processed, while no replayed or forged packet should create a sensor record or blockchain transaction.

3.2.6.3 Data Integrity Test Procedure

Data integrity refers to maintaining the accuracy and unchanged condition of sensor data while they are processed and stored. Most reviewed studies implemented integrity mechanisms but did not conduct direct tampering experiments. Studies [39], [42], and [44] used IPFS content identifiers or SHA-256 hashes, while Honar Pajooh et al. [43] computed checksums for off-chain IoT data and stored them in Hyperledger Fabric for later verification. Other systems relied on LoRaWAN message integrity codes or blockchain ledger immutability [40], [41], [46], [48]. Homaei et al. [47] conducted a more direct assessment by inserting altered error codes before blockchain recording and modifying blockchain files after storage. The abnormal records were examined before acceptance, while changes made to stored blockchain files were rejected because their hashes no longer agreed with the other validator records.

The present study will adapt the controlled tampering approach of Homaei et al. [47] and the checksum-verification design of Honar Pajooh et al. [43]. Two tests will be conducted: application-layer data alteration and post-storage database tampering. The first will determine whether a sensor record changed during Node-RED processing can be detected before it enters the operational database and blockchain. The second will determine whether a PostgreSQL/TimescaleDB record changed after its original hash has been committed to Hyperledger Fabric can be detected through hash comparison.

Testing will be performed on an isolated network using synthetic agricultural sensor readings. Each test will include 10 unchanged control records and 10 deliberately altered records, resulting in 40 total test attempts. Before testing, the PostgreSQL/TimescaleDB database will be backed up, timestamps will be synchronized, and logging will be enabled in Node-RED, PostgreSQL/TimescaleDB, and Hyperledger Fabric.

Preparation of the Integrity Record

For every sensor reading, Node-RED will create a canonical record containing the test identifier, device identifier, frame counter, sensor type, sensor value, measurement unit, and event timestamp. The fields will be placed in a fixed order, timestamps will use the same UTC format, and numerical values will use consistent decimal precision. This standardization is necessary because differences in field order, spacing, or numerical formatting can produce different hashes even when the actual information has not changed.

Node-RED will generate a SHA-256 hash from the canonical record. The complete sensor record will be stored in PostgreSQL/TimescaleDB, while its record identifier, SHA-256 hash, device identifier, and timestamp will be submitted to Hyperledger Fabric. The same canonicalization procedure will be used whenever a stored record is verified.

Table 8. Data Integrity Test Design

Test area	Test condition	Trials	Main measures	Expected result	Evidence collected
Application-layer integrity	Sensor record passes through Node-RED without alteration	10	Hash match, acceptance decision and verification time	Initial and processed hashes match; record is accepted	Node-RED logs, database record and Fabric transaction
	Sensor value is	10	Tamper	Hash mismatch	Original and

	altered after the initial hash is generated but before storage		detection, false acceptance and unauthorized storage	is detected; record is blocked or quarantined	processed hashes, Node-RED warning and absence of valid database/Fabric record
Post-storage integrity	Stored database record remains unchanged	10	Hash match, false-positive result and verification time	Database hash matches the Fabric hash	PostgreSQL record, Fabric hash and verification log
	Stored sensor value is changed after its original hash is committed to Fabric	10	Tamper detection, false negative and detection time	Recomputed database hash does not match the Fabric hash	Original Fabric hash, modified database value and integrity alert

Application-Layer Data-Alteration Procedure

A registered sensor will transmit a synthetic reading through LoRaWAN and ChirpStack to MQTT. When Node-RED receives the message, it will create the canonical record and calculate its initial SHA-256 hash. The record will then pass through the normal validation and processing flow.

For the control condition, the sensor value will remain unchanged. Node-RED will calculate the hash again immediately before database storage. The initial and final processing hashes should match, allowing the record to be stored in PostgreSQL/TimescaleDB and its hash to be submitted to Hyperledger Fabric.

For the alteration condition, a controlled test function in Node-RED will modify one sensor value after the initial hash has been generated. For example, a soil-moisture value of 35% may be changed to 45%. The device identity, frame counter, and timestamp will remain unchanged so that the test represents unauthorized alteration of the reading rather than the creation of a different record.

Node-RED will calculate a second hash from the altered record and compare it with the initial hash. A mismatch should cause the record to be marked as altered. It will be placed in a separate test or quarantine log and must not be stored as a valid operational reading or submitted to Hyperledger Fabric as a valid sensor transaction. Ten unchanged records and ten altered records will be tested.

Post-Storage Database-Tampering Procedure

Ten valid sensor readings will first be processed, stored in PostgreSQL/TimescaleDB, and committed to Hyperledger Fabric using their original SHA-256 hashes. Each record will have a unique test identifier linking the database entry to its corresponding Fabric transaction.

For the control condition, the integrity-verification function will retrieve each unchanged database record, convert it into the defined canonical format, recompute its SHA-256 hash, and compare the result with the hash retrieved from Hyperledger Fabric. All ten unchanged records should produce matching hashes.

For the tampering condition, a temporary database account created only for the controlled security test will be used to change the sensor value of each record after its original hash has been committed to Fabric. The blockchain record will not be changed. Immediately after each modification, the integrity-verification function will be executed.

The function will retrieve the modified PostgreSQL record, recompute its hash, and compare it with the original hash stored in Hyperledger Fabric. Because the sensor value has changed, the hashes should not match. The system should mark the record as altered, produce an integrity warning, and identify its record identifier, device identifier, verification time, database hash, and

blockchain hash. After the experiment, the modified records will be restored from the backup.

Measures and Data Analysis

For every trial, the researcher will record the expected integrity condition, actual verification result, original hash, recomputed hash, verification time, and whether the altered record entered the valid database or blockchain flow. The following measures will be used:

- **Integrity verification rate:** percentage of unchanged records correctly confirmed as valid.
- **Tamper-detection rate:** percentage of altered records correctly identified through a hash mismatch.
- **False-positive count:** unchanged records incorrectly identified as altered.
- **False-negative count:** altered records incorrectly reported as valid.
- **Unauthorized storage count:** altered application-layer records stored as valid operational records.
- **Verification time:** time from the start of the hash comparison until the system reports a match or mismatch.

Frequencies and percentages will be used to report integrity decisions. Verification time will be summarized using the mean and standard deviation. Results will be reported separately for application-layer alteration and post-storage tampering because the first test examines protection before valid storage, while the second examines verification after storage.

The security requirement will be zero false negatives and zero unauthorized storage. Every unchanged record should produce a matching hash, while every deliberately altered record should produce a hash mismatch. The test verifies whether data remain unchanged after the first hash is generated; it

does not prove that the original sensor reading was physically accurate before entering the system.

3.2.6.4 Traceability Test Procedure

Traceability refers to the ability of the prototype to identify the origin and complete processing history of every sensor reading. A traceable record should show which device produced the reading, when it was transmitted and processed, where the complete record was stored, and which Hyperledger Fabric transaction contains its hash and metadata. Previous studies commonly demonstrated traceability through blockchain histories and record queries rather than through direct attack experiments. Keo et al. [42] recorded the movement and history of agricultural batches, Honar Pajooh et al. [43] stored information about data ownership, submission time, storage location, and related records, while Yang et al. [44] used unique codes to retrieve complete product histories. Other studies used Fabric historian records, blockchain audit logs, and transaction histories to trace sensor readings and system activities [46]–[48]. However, most did not report a separate traceability success rate or evaluate whether every stored record could be completely and correctly reconstructed.

The present study will evaluate traceability through two tests: an individual sensor-record trace test and a device-history reconstruction test. The first will determine whether one sensor reading can be followed from its LoRaWAN source to PostgreSQL/TimescaleDB and Hyperledger Fabric. The second will determine whether a sequence of readings from one device can be retrieved in the correct chronological order with complete links between the operational database and blockchain records.

Testing will be conducted on the isolated prototype network using synthetic agricultural sensor readings. The individual-record test will use 10

readings, while the device-history reconstruction test will use 10 sequences containing five consecutive readings each. This will produce 20 traceability trials involving 60 sensor records. Logging will be enabled in ChirpStack, Mosquitto MQTT, Node-RED, PostgreSQL/TimescaleDB, and Hyperledger Fabric.

Preparation of Traceability Records

Node-RED will assign a unique `trace_id` to every accepted sensor reading. The trace identifier will remain attached to the record as it passes through the processing, storage, and blockchain-verification layers. The following information will be retained:

- trace identifier;
- device EUI;
- LoRaWAN frame counter;
- sensor type, value, and unit;
- sensor or event timestamp;
- gateway identifier;
- PostgreSQL record identifier;
- SHA-256 hash;
- Hyperledger Fabric transaction identifier; and
- blockchain commit timestamp or block number.

The complete sensor reading and related operational information will be stored in PostgreSQL/TimescaleDB. The trace identifier, hash, device identity, timestamp, and essential metadata will be recorded in Hyperledger Fabric. The common trace identifier will allow the database record to be linked directly to its blockchain transaction.

Table 9. Traceability Test Design

Test area	Test condition	Trials	Main measures	Expected result	Evidence collected
Individual sensor-record trace	Retrieve one selected reading from its LoRaWAN source through the database and blockchain record	10	Retrieval success, field completeness, database–blockchain linkage, and retrieval time	Every reading is located with complete and correctly linked information	ChirpStack event, Node-RED log, PostgreSQL record, Fabric transaction, and hash
Device-history reconstruction	Retrieve five consecutive readings from one device and reconstruct their processing history	10 sequences	Chronological accuracy, missing records, duplicate records, linkage accuracy, and reconstruction time	All readings are returned in the correct order without missing or duplicate records	PostgreSQL query results, Fabric history query, transaction IDs, timestamps, and hashes

Individual Sensor-Record Trace Procedure

The registered sensor will transmit 10 synthetic readings through LoRaWAN. Each reading will contain a known test value and will be assigned a unique trace identifier when received by Node-RED. The researcher will record the device EUI, frame counter, event timestamp, sensor value, and gateway information from ChirpStack and Node-RED.

After the reading has been processed, the researcher will use the trace identifier to retrieve its complete record from PostgreSQL/TimescaleDB. The corresponding Hyperledger Fabric transaction will then be retrieved using the transaction identifier stored with the database record.

For each trial, the fields recorded at the LoRaWAN and application layers will be compared with the database and blockchain information. The test will determine whether the device identity, frame counter, sensor value, timestamp, record identifier, hash, and Fabric transaction are available and correctly connected. A trial will be considered successful only when the

complete path of the reading can be reconstructed without an incorrect or missing link.

Device-History Reconstruction Procedure

The registered sensor will produce 10 separate sequences, with each sequence containing five consecutive readings transmitted at fixed intervals. Each record will receive a unique trace identifier and will be stored in PostgreSQL/TimescaleDB with its corresponding Fabric transaction identifier.

For every sequence, the researcher will query PostgreSQL/TimescaleDB using the device EUI and defined time range. The retrieved records will be arranged according to their timestamps and frame counters. The related blockchain records will then be obtained through their Fabric transaction identifiers or chaincode history query.

The reconstructed sequence will be compared with the original transmission log. The researcher will verify whether all five readings are present, appear in the correct order, contain the correct values and timestamps, and remain linked to their corresponding blockchain hashes and transactions. Missing, duplicate, incorrectly ordered, or incorrectly linked records will be recorded as traceability errors.

Measures and Data Analysis

For every trial, the researcher will record whether the expected records were retrieved, whether the required fields were complete, whether the database and blockchain records were correctly linked, and how long the retrieval or reconstruction required. The following measures will be used:

- **Trace retrieval success rate:** percentage of individual records successfully retrieved from both PostgreSQL/TimescaleDB and Hyperledger Fabric.
- **Record completeness rate:** percentage of required traceability fields available and correct.
- **Database–blockchain linkage rate:** percentage of database records correctly connected to their corresponding blockchain hashes and transaction identifiers.
- **Chronological-order accuracy:** percentage of readings reconstructed in the correct sequence.
- **Missing-record count:** number of transmitted readings that could not be retrieved.
- **Duplicate-record count:** number of readings appearing more than once in the reconstructed history.
- **Trace retrieval time:** time required to retrieve the complete history of an individual record.
- **History reconstruction time:** time required to retrieve and arrange one sequence of device readings.

Frequencies and percentages will be used to summarize trace retrieval, completeness, linkage, chronological order, and missing or duplicate records. Retrieval and reconstruction times will be reported using the mean and standard deviation. The two tests will be reported separately because the individual-record test examines the traceability of one reading, while the history-reconstruction test evaluates the continuity of several readings over time.

The traceability requirement will be complete retrieval and correct linkage of all tested records. Every sensor reading should be connected to its original device information, PostgreSQL record, SHA-256 hash, and

Hyperledger Fabric transaction. No expected record should be missing, duplicated, or placed in the wrong chronological order.

3.2.6.5 DoS or flooding Test Procedure

A Denial-of-Service (DoS) or flooding attack occurs when a large number of requests or messages are sent within a short period to consume processing power, memory, bandwidth, or other system resources. The attack attempts to slow down the system or prevent legitimate sensor readings from being processed. Among the reviewed studies, Hou et al. [40] conducted the clearest DoS-like experiment by generating traffic from authorized and unauthorized devices. Invalid packets were rejected after failing message-integrity verification, while system throughput, CPU utilization, and network bandwidth were measured. Their results showed whether legitimate traffic could continue while invalid traffic increased. The present study will follow the same mixed-traffic approach but adapt it to the MQTT and application layers of the proposed prototype. Radio-frequency jamming will not be included because it requires specialized equipment and may interfere with other wireless users.

Two controlled flooding tests will be conducted: an MQTT connection-flooding test and an invalid application-message flooding test. Both tests will be performed on an isolated local network using a separate laptop as the traffic generator. A legitimate agricultural sensor will continue sending one reading every 15 seconds throughout each test. This will allow the researcher to determine whether valid sensor readings remain available while invalid traffic is being received.

Each test will include three conditions: normal traffic, moderate flooding, and high flooding. Every condition will run for five minutes and will be repeated three times. Therefore, the two tests will involve 18 experimental

runs. A five-minute recovery period will be provided after every run to allow the system to clear pending requests and return to normal operation.

Table 10. DoS or Flooding Test Design

Test area	Traffic condition	Invalid traffic rate	Expected result
MQTT connection flooding	Normal traffic	No invalid attempts	Legitimate sensor messages are processed normally
	Moderate flooding	10 invalid connections per second	Invalid connections are rejected while legitimate messages continue
	High flooding	50 invalid connections per second	System remains available and rejects invalid connections without serious loss of legitimate data
Invalid application-message flooding	Normal traffic	No invalid messages	Legitimate messages are validated and stored
	Moderate flooding	10 invalid messages per second	Invalid messages are rejected and legitimate messages continue through the system
	High flooding	50 invalid messages per second	Invalid messages do not create database or blockchain records, and the system recovers after the test

Note(s): Duration: 5 minutes, Repetitions: 3

The proposed traffic rates will first be checked during a short pilot test. When a selected rate causes an immediate system failure or does not produce a measurable increase in resource use, it may be adjusted before the main experiment. The final traffic rates will remain fixed across all three repetitions.

For all test conditions, one registered sensor will continue transmitting one legitimate reading every 15 seconds, producing approximately 20 valid readings during each five-minute run. Under normal traffic, no invalid requests will be generated. Moderate flooding will introduce 10 invalid connection attempts or messages per second, equivalent to approximately 3,000 invalid requests per run, while high flooding will introduce 50 per second, equivalent to approximately 15,000 invalid requests per run. Each condition will be repeated three times using the same traffic rates to support consistent comparison of system performance.

MQTT Connection-Flooding Procedure

The MQTT connection-flooding test will determine whether repeated connection attempts using incorrect credentials can affect the delivery of legitimate sensor messages. Anonymous MQTT access will remain disabled, and Mosquitto authentication will be enabled.

During the normal condition, the registered agricultural sensor will transmit readings through ChirpStack to Mosquitto MQTT, Node-RED, PostgreSQL/TimescaleDB, and Hyperledger Fabric. No invalid connection attempts will be generated. This condition will establish the normal delivery rate, latency, and resource use of the prototype.

During the moderate and high flooding conditions, a load-generation script on the test laptop will repeatedly attempt to connect to Mosquitto using an incorrect username or password. The script will generate 10 invalid connection attempts per second during the moderate condition and 50 attempts per second during the high condition. The legitimate sensor will continue transmitting throughout the test.

Mosquitto should reject the invalid connections while continuing to accept messages from ChirpStack and Node-RED. The researcher will examine the Mosquitto logs to count rejected connections and will check Node-RED, PostgreSQL/TimescaleDB, and Hyperledger Fabric to determine whether all legitimate readings were processed.

Invalid Application-Message Flooding Procedure

The invalid application-message flooding test will determine whether Node-RED can reject excessive malformed or unauthorized messages without interrupting legitimate processing. A limited MQTT test account will be allowed to publish only to a controlled test topic connected to the same Node-RED validation flow. The account will not have access to the production ChirpStack topics.

During the normal condition, only legitimate sensor messages will pass through the processing flow. During the flooding conditions, the test laptop will publish invalid JSON messages containing missing fields, incorrect device identifiers, invalid timestamps, or values outside the expected data format. Moderate flooding will generate 10 invalid messages per second, while high flooding will generate 50 invalid messages per second.

Node-RED will validate the incoming messages. Invalid messages should be rejected or placed in a separate test log and must not be stored as valid records in PostgreSQL/TimescaleDB or submitted to Hyperledger Fabric. The legitimate sensor messages should continue to be processed during the flooding period.

After every run, the flood traffic will be stopped. The researcher will continue monitoring the system to determine how long it takes for CPU use, memory use, message processing, and latency to return to their normal levels.

Measures and Data Analysis

The following measures will be collected during every experimental run:

- **Invalid-traffic rejection rate:** percentage of invalid connections or messages correctly rejected.
- **Legitimate-message delivery rate:** percentage of expected sensor readings successfully stored during the test.
- **End-to-end latency:** time from legitimate sensor transmission until database storage.
- **Valid-message throughput:** number of legitimate sensor readings successfully processed per unit of time.
- **CPU utilization:** average and maximum Raspberry Pi processor use during each condition.
- **Memory utilization:** average and maximum memory use during each condition.
- **Network bandwidth:** amount of network traffic received and transmitted during the test.
- **Service availability:** whether Mosquitto, Node-RED, PostgreSQL/TimescaleDB, and Hyperledger Fabric remained operational.
- **Unauthorized record count:** number of invalid messages incorrectly stored in the database or blockchain.
- **Recovery time:** time from the end of flooding until latency and resource use return to the normal range.

Mosquitto and Node-RED logs will provide the counts of received and rejected requests. PostgreSQL/TimescaleDB and Hyperledger Fabric will be checked for legitimate and unauthorized records. Raspberry Pi CPU, memory, and network utilization will be collected using system and Docker resource-

monitoring logs. Grafana may be used to display the measurements throughout each test.

The mean and standard deviation will be reported for latency, throughput, CPU utilization, memory utilization, bandwidth, and recovery time. Frequencies and percentages will be reported for legitimate-message delivery, invalid-traffic rejection, service availability, and unauthorized records. Results from the normal, moderate-flooding, and high-flooding conditions will be compared to determine whether increasing invalid traffic significantly affects system operation.

The principal security requirements will be that invalid traffic is rejected, no invalid message creates a valid database or blockchain record, and the prototype continues processing legitimate sensor readings. Any service crash, unauthorized record, or sustained failure to process legitimate messages will be classified as a flooding-related security failure.

3.2.6.6 Resilience Test Procedure

System resilience refers to the ability of the prototype to continue its essential functions during a disruption and return to normal operation without losing or duplicating data. Hou et al. [40] discussed resilience through the reduction of single points of failure, but they did not directly perform a gateway or server-interruption experiment. Homaei et al. [47] conducted a practical resilience test by interrupting Internet connectivity for two hours per day over one week. Their gateway temporarily retained the sensor readings and submitted the pending information after connectivity was restored. They evaluated whether records were lost and whether the delayed transactions caused blockchain conflicts.

Following this approach, the present study will conduct one **Internet-connectivity interruption and recovery test**. The test will determine whether the prototype continues collecting, processing, storing, displaying, and verifying agricultural sensor readings while its external Internet connection is unavailable. Only the Internet or wide-area network connection will be disconnected; the LoRaWAN gateway, Raspberry Pi, local-area network, and locally hosted services will remain powered and operational.

Each experimental run will include a 30-minute normal period, a 60-minute Internet-interruption period, and a 30-minute recovery period. The sensor will transmit one reading every 15 seconds throughout the two-hour run. Approximately 120 readings are expected before the interruption, 240 during the interruption, and 120 after reconnection, giving approximately 480 readings per run. The complete procedure will be repeated

Table 11. Resilience Test Design

Test condition	Duration	Expected readings	Expected system behavior	Evidence collected
Normal operation before interruption	30 minutes	Approximately 120	Readings are received, processed, stored, displayed, and verified normally	ChirpStack, MQTT, Node-RED, PostgreSQL/TimescaleDB, Grafana, and Fabric records
Internet interruption	60 minutes	Approximately 240	Local collection, processing, storage, visualization, and blockchain verification continue without Internet access	Local service logs, database records, Fabric transactions, and system-resource logs
Recovery after reconnection	30 minutes	Approximately 120	Internet access returns without service failure, missing records, duplicate records, or incorrect record order	Connectivity logs, database and blockchain queries, timestamps, and transaction records

Internet-Connectivity Interruption and Recovery Procedure

Before each run, the researcher will confirm that ChirpStack, Mosquitto MQTT, Node-RED, PostgreSQL/TimescaleDB, Grafana, and Hyperledger Fabric are operating normally. The sensor will then transmit readings every 15 seconds for 30 minutes to establish the baseline condition. The number of transmitted, received, stored, and blockchain-recorded readings will be documented.

After the baseline period, the Raspberry Pi's external Internet connection will be disabled for 60 minutes while its local network connection remains active. The sensor and gateway will continue operating normally. During the interruption, the researcher will check whether readings continue to appear in ChirpStack, MQTT, Node-RED, PostgreSQL/TimescaleDB, Grafana, and Hyperledger Fabric. The local system should not require access to a remote cloud platform to perform these functions.

After 60 minutes, Internet connectivity will be restored. The prototype will be monitored for another 30 minutes to determine whether all services remain operational and whether normal connectivity returns without manual data correction. Database records, blockchain transactions, timestamps, frame counters, and trace identifiers will be compared with the original sensor-transmission log.

Measures and Data Analysis

The following measures will be collected for each run:

- **Local service availability:** whether ChirpStack, MQTT, Node-RED, PostgreSQL/TimescaleDB, Grafana, and Hyperledger Fabric remained operational during the interruption.
- **Data-delivery rate:** percentage of expected sensor readings successfully received and stored.

- **Data-loss count:** number of transmitted readings not found in the database or blockchain records.
- **Duplicate-record count:** number of readings recorded more than once.
- **Database–blockchain consistency:** percentage of database records correctly linked to their corresponding blockchain hashes and transactions.
- **End-to-end latency:** time from sensor transmission to PostgreSQL/TimescaleDB storage during normal, interrupted, and recovery periods.
- **Recovery time:** time from the restoration of Internet connectivity until external connectivity and related services return to their normal condition.
- **Chronological accuracy:** whether records remain in the correct sequence based on timestamps and frame counters.

Frequencies and percentages will be used to report service availability, delivery rate, missing records, duplicates, and database–blockchain consistency. End-to-end latency and recovery time will be summarized using the mean and standard deviation. Results from the normal, interruption, and recovery periods will be compared separately.

The resilience requirement will be that essential local monitoring functions continue during the Internet interruption. No expected sensor record should be lost, duplicated, or incorrectly linked, and the system should return to normal connectivity without corrupting the PostgreSQL/TimescaleDB or Hyperledger Fabric records. This test evaluates resilience to Internet loss only and does not represent resilience to Raspberry Pi power failure or complete gateway failure.

3.3 Ethical Consideration

This study involved the development and testing of an IoT and blockchain-based prototype and did not collect information from human participants. The sensor gathered only environmental readings, such as temperature and humidity. No personal, medical, financial, or identifying information was included in the system.

The prototype was operated only for research purposes. Field installation and testing were conducted with the knowledge and permission of the concerned plantation management. The equipment was placed and operated in a way that did not interfere with normal plantation activities, damage crops, or create safety risks for farm personnel.

All device identifiers, access keys, login credentials, server addresses, and blockchain identities were handled securely. Sensitive technical information was not publicly disclosed to prevent unauthorized access to the prototype. The collected sensor records were used only for system development, verification, and performance testing.

The tamper-detection test was performed in a controlled environment. Only selected test records stored in PostgreSQL were intentionally modified, and no actual plantation records used for operational decisions were affected. The test was limited to determining whether the prototype could identify post-storage changes through hash mismatches.

The results were reported honestly, including successful and failed transmissions, performance measurements, and the limitations of the prototype. Since this dissertation expands an earlier published paper, the original publication will be properly cited and acknowledged. Similarity in the system, data, and results will be disclosed to maintain academic integrity and avoid presenting previously published work as entirely new.